企画書・実装書

koshi ターミナルネイティブ分散型 SNS

サーバー不要の P2P 対話ボード、設計から導入まで。ed25519 認証・P2P メッシュ・Nostr 連携を解説。

game_ryo (ryopc)
v3.1.4 · 2026.10.18
MIT License · https://koshi.js.org

目次

01	エグゼクティブサマリー	3
02	背景と課題定義	4
03	プロダクト概要とコア機能	5
04	システムアーキテクチャ	6
05	技術実装: 認証、暗号化、Nostr 連携	7
06	導入・操作ガイド	8
07	セキュリティ、運用、ロードマップ	9
08	付録: API リファレンス、用語集、参考資料	10

01 · エグゼクティブサマリー

エグゼクティブサマリー

koshi はサーバーを立てずに PC 一台で即座に始められるターミナルネイティブの分散型 SNS。ed25519 鍵認証と P2P ゴシップメッシュで投稿・ダイレクトメッセージ・プロフィールを仲介なしに同期し、Nostr プロトコルとも連携する。

核心 Takeaways

- Serverless モードがデフォルト。PostgreSQL や外部サーバーは不要で、SQLite + hypercore + hyperswarm がローカルで動く
- ed25519 鍵ベース認証と x25519 エンドツーエンド暗号化 DM で、投稿とプライベートメッセージの正当性と機密性を担保
- Nostr 互換(nsec/npub キー、リレーへのプッシュ / プル)で既存の分散型ソーシャルグラフと相互運用可能
- `kb serve` 一コマンドで REST API、WebSocket、Web UI、P2P メッシュが同時に立ち上がり、ブラウザからも操作できる

本ドキュメント回答の問題

koshi はどのような分散型 SNS なのか? サーバーなしでどうデータを同期するのか? どうやって導入・運用するのか?

02 · 背景と課題定義

背景と課題定義

既存の集中型 SNS はプラットフォーム事業者による検閲・アカウント凍結・データ独占のリスクを抱え、ユーザーが自分のデータとアイデンティティを完全にコントロールできない。分散型・サーバーレスの代替手段として、技術者がすぐに導入・運用できるツールが求められている。

現状

現代の主要 SNS はすべて集中型サーバーに依存し、運営会社の判断一つでコンテンツが削除・非表示化され、アカウントが凍結されるリスクが常にある。ユーザー投稿やダイレクトメッセージはプラットフォーム事業者のデータベースに預託され、エクスポートや移行も自由ではない。

核心課題

分散型 SNS の既存選択肢（Mastodon、Nostr など）はサーバー構築やリレー運用などの専門知識を必要とし、一般の技術者が「今すぐ使い始める」にはハードルが高い。サーバー不要で PC 一台から始められる、ターミナルとブラウザの両方で動く分散型 SNSのニーズが高い。

> 「サーバーを立てずに、PC 一台ですぐ始められる」ことが、分散型 SNS の普及を左右する鍵となる。

本ドキュメントの位置づけ

本ドキュメントは、MIT ライセンスのオープンソースプロダクト「koshi」の設計思想、アーキテクチャ、実装詳細、導入手順をまとめた企画書・実装書である。読者は本書を読むことで、koshi がどのような分散型 SNS なのか、どう導入・運用すべきかを判断できる。

03 · プロダクト概要とコア機能

プロダクト概要とコア機能

koshi は ed25519 鍵ベースの分散認証、P2P ゴシップによるデータ同期、Nostr プロトコル連携、ターミナル CLI と Web UI の二重インターフェースを備えた分散型 SNS。サーバーレスモードがデフォルトであり、PostgreSQL なしで動作する。

コア機能

koshi の核心機能は、投稿(ポスト)、フォロー、エンドツーエンド暗号化 DM、プロフィール編集、マルチアカウント管理、Nostr 連携、P2P メッシュ同期、Web UI である。すべて `kb` という単一 CLI コマンドから操作でき、専用の管理画面やサーバー構築を必要としない。

分散認証

認証は ed25519 公開鍵暗号に基づき、ユーザーは自身のキーペアを保有する。パスワードベースの集中管理ではなく、鍵署名による本人確認を行い、アカウントの所有権をユーザー自身が握る。

P2P データ同期

投稿と DM は hypercore の追加専用ログに追記され、hyperswarm による P2P トピック(グローバルボード・個別ユーザートピック)を通じて接続ピア間に自動複製される。サーバーを介さずにデータがメッシュ状に同期される。

エンドツーエンド暗号化 DM とチャット

DM は ed25519 キーから導出した x25519 鍵でエンドツーエンド暗号化され、送信者と受信者のみが復号できる。WebSocket を用いた LINE 風バブルチャット UI をターミナル上で提供し、リアルタイム対話を可能にする。

04 · システムアーキテクチャ

システムアーキテクチャ

koshi は SQLite ローカルデータベースと hypercore / corestore / hyperswarm による P2P メッシュを組み合わせたサーバーレスアーキテクチャを採用。リモートモードでは PostgreSQL バックエンドの koshi サーバーに接続でき、モード切替は設定ファイルの serverUrl フィールドで行う。

サーバーレスモード(デフォルト)

サーバーレスモード(デフォルト)では、すべてのデータは `~/.config/koshi/local.db` の SQLite に保存される。外部データベースやサーバー構築は不要で、`npm install -g @ryopc/koshi` 後の `kb register <ユーザー名>` ですぐに利用を開始できる。

P2P メッシュ層

P2P 層は hypercore(追加専用ログ)、corestore(複数 hypercore インスタンスの管理)、hyperswarm(P2P ネットワーク発見と接続管理)で構成される。グローバルボードトピック(koshi:v1:board)に投稿とプロフィール発表がゴシップ配信され、個別ユーザートピック(koshi:v1:user:)に DM が届く。

P2P ブリッジとリモートモード

P2P ブリッジが P2P 裏のデータとローカル SQLite を reconciles し、Web UI や CLI は SQLite 経由でデータを読み書きする。P2P は Node.js 環境(CLI または kb serve)で動作し、ブラウザは REST API と WebSocket 経由でローカルノードにアクセスする。

リモートモード(`kb connect`)では、PostgreSQL バックエンドを持つ koshi サーバーに接続し、REST API 経由で登録・投稿・リアルタイムストリームを利用する。`kb connect --reset` でサーバーレスモードに戻せる。

図 1: koshi のサーバーレス P2P アーキテクチャ(architecture)。SQLite と P2P メッシュがローカルノードで動作し、ブラウザは WebSocket 経由で接続する。

05 · 技術実装: 認証、暗号化、NOSTR 連携

技術実装: 認証、暗号化、Nostr 連携

koshi は ed25519 署名による投稿・DM の正当性担保、x25519 による DM エンドツーエンド暗号化、JWT 認証(24 時間有効)、Nostr プロトコル(nsec/npub キー、リレー連携)を実装し、分散型ソーシャルのセキュリティ要件を満たす。

認証層と署名

認証層は ed25519 キーペアを用い、すべての投稿と DM に ed25519 署名を付与することで改ざんを検知する。tweetnacl(ed2curve 経由)で ed25519 キーを x25519 キーに変換し、nacl.box によるシールボックスで DM を暗号化する。

JWT 認証とレート制限

サーバーレスモードでは JWT は HMAC-SHA256 で署名され、24 時間で失効する。認証エンドポイントは IP あたり 10 req/分のレート制限を設け、Helmet によるセキュリティヘッダ、CORS、SQL インジェクション対策(パラメータ化クエリ)を施す。

Nostr 連携

Nostr 連携により、ユーザーは nsec/npub キーを生成・管理でき、`kb nostr push` で koshi 投稿を Nostr リレー(wss://relay.damus.io など)にクロス投稿、`kb nostr pull` で Nostr イベントを koshi フィードに取り込める。既存の Nostr グラフと相互運用可能。

設定保護とマルチアカウント

設定ファイル(`~/.config/koshi/config.json`)のパーミッションは 0600(所有者のみ読み書き)に設定され、認証情報や Nostr キーが保護される。マルチアカウント対応により、複数の koshi アイデンティティを単一 CLI から切り替えて管理できる。

06 · 導入・操作ガイド

導入・操作ガイド

koshi の導入は `npm install -g @ryopc/koshi` と `kb register <ユーザー名>` のニステップで完了し、`kb serve` でローカルノード（REST＋WebSocket＋Web UI＋P2P）を即座に立ち上げられる。Docker スタンドアロンイメージも提供され、ワンコマンドで自己ホスト可能。

インストール

インストール方法は npm グローバルインストールとソースからのビルドの二通り。npm では `npm install -g @ryopc/koshi`、ソースでは `git clone` 後の `npm install` と `npm link` で `kb` コマンドを利用可能にする。Node.js 20.0.0 以上が必要（better-sqlite3 のネイティブモジュールサポートのため）。

主要 CLI コマンド

CLI の主要コマンドは `kb register`（アカウント作成）、`kb post`（投稿）、`kb feed`（フィード表示）、`kb follow` / `kb unfollow`（フォロー管理）、`kb dm`（DM 送信）、`kb chat`（リアルタイム暗号化チャット）、`kb edit-profile`（プロフィール更新）、`kb search`（ユーザー検索）、`kb realtime`（リアルタイムイベントストリーム）などである。

サーバーモードと Docker

サーバーモードでは `kb serve [--port=3000] [--daemon]` でローカルノードを起動し、`http://localhost:3000` で REST API と Web UI にアクセスできる。`kb serve --daemon` でバックグラウンド実行、`kb serve --stop` で停止。`kb connect` でリモートサーバーに切り替え、`kb connect --reset` でサーバーレスに戻る。

Docker スタンドアロンイメージ（`ghcr.io/ryopc/koshi:standalone-v3.1.4`）は PostgreSQL なしで動作し、`docker run -d -p 3000:3000 -v $HOME/.config/koshi:/home/koshi/.config/koshi ghcr.io/ryopc/koshi:standalone-v3.1.4` で自己ホストノードを即座に立ち上げられる。Render.com + Neon.tech 構成での PostgreSQL バックエンドデプロイもドキュメント化されている。

07 · セキュリテイ、運用、ロードマップ

セキュリテイ、運用、ロードマップ

koshi は ed25519 署名、x25519 E2E 暗号化 DM、JWT 認証、レート制限、SQL インジェクション対策、Helmet セキュリテイヘツダ、設定ファイル 0600 パーミツシヨンなど、多層的なセキュリテイ対策を講じる。ロードマツプは P2P メツシユの安定化と Web UI 機能拡充に向かう。

セキュリテイ対策

セキュリテイ対策として、全投稿と DM に ed25519 署名を付与し改ざんを検知。DM は x25519 エンドツーエンド暗号化。JWT は 24 時間で失効。レート制限は認証エンドポイントあたり 10 req/分。Helmet でセキュリテイヘツダを有効化し、CORS は展開ごとに設定可能。SQL インジェクションはパラメータ化クエリで防止。

運用とバツクアツプ

運用面では、サーバーレスモードでは外部サービスへの依存がなく、ログとデータはすべてローカルに保存される。バツクアツプは `~/.config/koshi/local.db` と `~/.config/koshi/config.json` をコピーするだけで完了。マルチアカウントとアカウント自己削除(`kb delete --force`)もサポート。

管理機能

管理機能として、最初に登録されたユーザーがローカルモードの管理者となり、ユーザー一覧・ユーザー削除・管理者付与・剥奪(`kb admin`)を行える。リモートサーバーでは `ADMIN_USERNAME` 環境変数または直接 DB 更新で管理者を指定する。

ロードマツプ

ロードマツプは v3.1.4(Serverless P2P メツシュ、アカウント自己削除、Nostr リレー修正)を経て、P2P メツシュの安定化、Web UI の機能拡充、モバイル端末からのアクセス改善、より大規模なメツシュネツトワークへの対応を進める。MIT ライセンスにより、コミュニテイによる改良とフォークを歓迎する。

08 · 付録

付録: API リファレンス、用語集、参考資料

付録では koshi の REST API エンドポイント一覧、主要用語の定義、参考となる外部資料をまとめる。実装者や運用者が koshi を統合・拡張する際の参照資料とする。

A. REST API エンドポイント

REST API は認証(/api/auth/register、/api/auth/login)、ユーザー(/api/users/:username、/api/users/me、フォロー関連)、投稿(/api/posts/feed、/api/posts、/api/posts/:id)、DM(/api/dms、/api/dms/:userId、既読マーク、未読数)、管理(/api/admin/users 等)、P2P(/api/p2p/status、/api/p2p/peers、/api/p2p/posts、/api/p2p/sync 等)、WebSocket(/ws?token={jwt})、ヘルスチェック(/api/health)を提供する。

B. 用語集

ed25519: 公開鍵署名アルゴリズム。認証と署名に使用。

x25519: 鍵合意アルゴリズム。DM 暗号化に使用。

hypercore: 追加専用ログ。

corestore: hypercore ファクトリ。

hyperswarm: P2P 発見・接続管理。

Nostr: 分散型ソーシャルプロトコル。nsec/npub キー。

JWT: 認証トークン。24 時間有効。

C. 参考資料

- koshi README.md (v3.1.4) - https://github.com/ryopc/koshi
- @noble/ed25519 - https://github.com/paulmillr/noble-ed25519
- tweetnacl - https://tweetnacl.js.org/
- ed2curve - https://github.com/dominictarr/ed2curve
- hypercore / corestore / hyperswarm - https://github.com/hypercore-protocol
- nostr-tools - https://github.com/paulmillr/nostr-tools
- better-sqlite3 - https://github.com/WiseLibs/better-sqlite3
- Express / Helmet - https://expressjs.com/ / https://helmetjs.github.io/